Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЕТ
к лабораторной работе №1
на тему:
«КОНВЕРТЕР ЕДИНИЦ ИЗМЕРЕНИЯ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Минимум 3 разные категории единиц (расстояние, вес, валюта и т.д.).

2 Не менее 3 различных единиц в каждой категории.

3 Цифровая клавиатура, похожая на калькулятор, и два поля для исходного и преобразованного значений. Пользователь не должен иметь возможности изменять данные в этих полях с помощью стандартной системной клавиатуры.

4 Рекомендуется оформить клавиатуру и область с данными как отдельные фрагменты (Fragments) Android. Портретная ориентация: фрагменты расположены в два рядя (клавиатура под фрагментом с данными). Альбомная ориентация: фрагменты в одном ряду (слева – данные, справа – клавиатура).

5 Создайте build flavour «premium». Должна быть кнопка, которая будет менять местами исходные и преобразованные значения (и единицы измерения), и наоборот. Кнопка для копирования текущего значения в буфер обмена рядом с каждым полем ввода данных. Используйте source sets для определения различных макетов и файлов с кодом для каждого варианта (flavour).

В отчете по лабораторной работе представьте следующую диаграмму: Диаграмма объектов (Object Diagram).

2 ВЫПОЛНЕНИЕ РАБОТЫ

Разработано приложение "Конвертер единиц измерения" для платформы Android с использованием современного фреймворка Jetpack Compose. Приложение позволяет конвертировать значения между единицами измерения в трёх категориях: расстояние, вес и температура. Интерфейс адаптируется под портретную и альбомную ориентацию устройства. Интерфейс портретной ориентации в светлой теме представлен на рисунке 2.1.

Для портретной ориентации реализована вертикальная компоновка: область ввода данных расположена над цифровой клавиатурой. Для альбомной ориентации – горизонтальная: область данных слева, клавиатура справа.

Используются компоненты ConverterTextField, которые отображают значение, текущую единицу измерения и предоставляют выпадающий список для её смены. Реализована индикация активного (фокусированного) поля.

Реализован кастомный компонент CalculatorKeyboard, имитирующий калькулятор. Он включает цифры, точку для десятичных дробей, кнопки изменения знака (для температуры), удаления последнего символа (Backspace) и сброса (Clear). Поля ввода защищены от редактирования системной клавиатурой, так как весь ввод осуществляется только через эту кастомную клавиатуру.

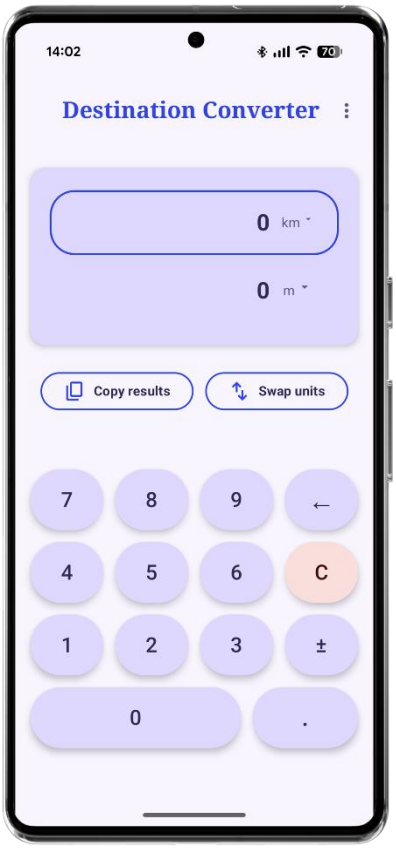

Рисунок 2.1 – Интерфейс портретной ориентации в светлой теме

Согласно условию, интерфейс портретной ориентации оформлен в два столбца, что представлено на рисунке 2.2

Рисунок 2.2 – Интерфейс альбомной ориентации в светлой теме

Приложение использует систему токенов Material Design 3 через MaterialTheme, который автоматически адаптирует цвета под текущую тему устройства. В файле Theme.kt определены цветовые схемы приложения.

Рассмотрим, в таком случае, как будет выглядеть приложение в тёмной теме, на рисунке 2.3.

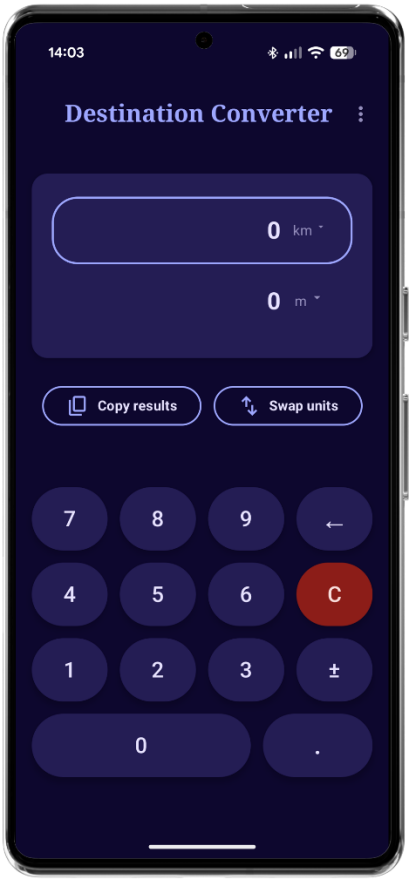

Рисунок 2.3 – Интерфейс портретной ориентации в тёмной теме

Соответственно, представим ниже, на рисунке 2.4, как будет выглядеть приложение в альбомной ориентации и тёмной теме.

Рисунок 2.4 – Интерфейс альбомной ориентации в тёмной теме

Приложение построено на паттерне MVVM (Model-View-ViewModel). ConverterViewModel, хранящим состояние интерфейса (текущий маршрут, значения полей, выбранные единицы измерения, фокус) и содержащим всю бизнес-логику обработки ввода и конвертации.

Реализована простейшая навигация между категориями конвертации (Routes) через выпадающее меню в TopAppBar.

Алгоритмы конвертации централизованы в companion object каждого подкласса MeasurementUnit. Преобразование выполняется через приведение к базовой единице (метры, килограммы, градусы Цельсия) и последующее вычисление целевого значения.

Приложение динамически определяет ориентацию устройства (LocalConfiguration.current.orientation) и перестраивает макет, используя различные компоновки (PortraitLayout, LandscapeLayout), что соответствует требованию о различном расположении областей интерфейса.

В качестве наглядного представления структуры основных объектов приложения и их связей в runtime разработана диаграмма объектов, рассмотреть её можно на рисунке 2.5.

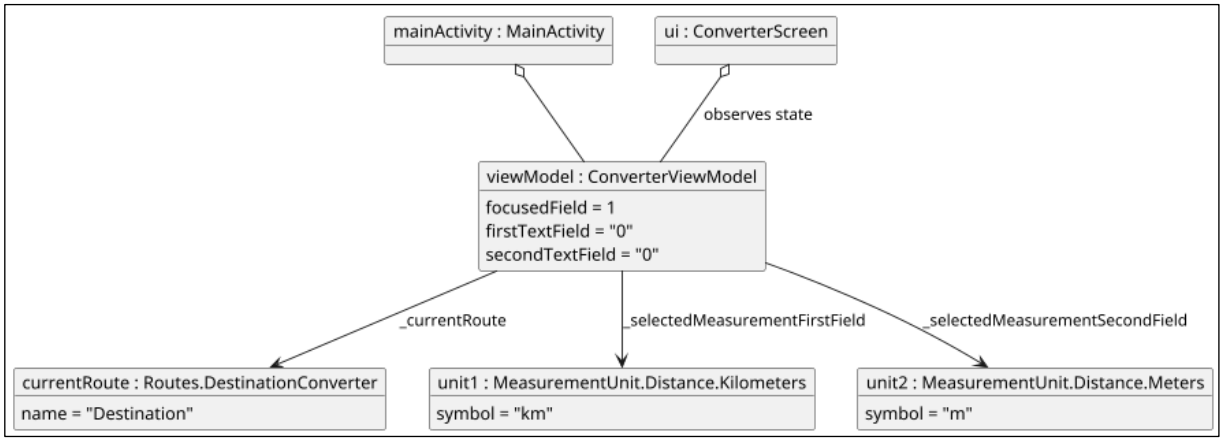

Рисунок 2.5 – Диаграмма объектов приложения

ВЫВОД

В рамках работы успешно разработано полнофункциональное приложение-конвертер, отвечающее всем поставленным функциональным требованиям. Приложение демонстрирует практическое использование Jetpack Compose для создания адаптивного UI, применение архитектуры MVVM для управления состоянием, реализацию кастомных компонентов.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация конвертера единиц измерения

```
sealed class MeasurementUnit(
    val symbol: String
) {
    sealed class Distance(
        symbol: String,
    ) : MeasurementUnit(symbol) {

        abstract fun fromMeters(value: Double): Double
        abstract fun toMeters(value: Double): Double

        object Meters : Distance("m") {
            override fun fromMeters(value: Double): Double = value

            override fun toMeters(value: Double): Double = value
        }

        object Kilometers : Distance("km") {
            override fun fromMeters(value: Double): Double = value * 0.001

            override fun toMeters(value: Double): Double = value * 1000
        }

        object Miles : Distance("mi") {
            override  fun  fromMeters(value:  Double):  Double  =  value  *
0.000621371

            override fun toMeters(value: Double): Double = value * 1609.344
        }

        companion object {
            val allUnits by lazy { listOf(Meters, Kilometers, Miles) }

            fun convert(value: Double, from: Distance, to: Distance): String {
                val inMeters = from.toMeters(value)
                return to.fromMeters(inMeters).to2DigitFormat()
            }
        }
    }

    sealed class Weight(
        symbol: String,
    ) : MeasurementUnit(symbol) {

        abstract fun fromKilograms(value: Double): Double
        abstract fun toKilograms(value: Double): Double

        object Kilograms : Weight("kg") {
            override fun fromKilograms(value: Double): Double = value
            override fun toKilograms(value: Double): Double = value
        }

        object Grams : Weight("g") {
            override fun fromKilograms(value: Double): Double = value * 1000
            override fun toKilograms(value: Double): Double = value * 0.001
        }
```

```
        object Pounds : Weight("lb") {
            override fun fromKilograms(value: Double): Double = value * 2.20462
            override fun toKilograms(value: Double): Double = value * 0.453592
        }

        companion object {
            val allUnits by lazy { listOf(Kilograms, Grams, Pounds) }

            fun convert(value: Double, from: Weight, to: Weight): String {
                val inKilograms = from.toKilograms(value)
                return to.fromKilograms(inKilograms).to2DigitFormat()
            }
        }
    }

    sealed class Temperature(
        symbol: String
    ) : MeasurementUnit(symbol) {

        abstract fun toCelsius(value: Double): Double
        abstract fun fromCelsius(value: Double): Double

        object Celsius : Temperature("°C") {
            override fun toCelsius(value: Double): Double = value
            override fun fromCelsius(value: Double): Double = value
        }

        object Fahrenheit : Temperature("°F") {
            override fun toCelsius(value: Double): Double = (value - 32) * 5 /
9
            override fun fromCelsius(value: Double): Double = (value * 9 / 5)
+ 32
        }

        object Kelvin : Temperature("K") {
            override fun toCelsius(value: Double): Double = value - 273.15
            override fun fromCelsius(value: Double): Double = value + 273.15
        }

        companion object {
            val allUnits by lazy { listOf(Celsius, Fahrenheit, Kelvin) }

            fun convert(value: Double, from: Temperature, to: Temperature):
String {
                val inCelsius = from.toCelsius(value)
                return to.fromCelsius(inCelsius).to2DigitFormat()
            }
        }
    }
}
```